

# Mini RFC: Connection Termination

April 5, 2022

**Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.**

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

**WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.**

## 1 Overview

An established QUIC connection can be terminated in one of three ways:

- idle timeout
- immediate close ([has been discussed in \*mini RFC: The QUIC Transport Protocol\*](#))
- stateless reset

An endpoint MAY discard connection state if it does not have a validated path on which it can send packets.

Check *mini RFC: Path Validation* for more details.

## 2 Idle Timeout

If a `max_idle_timeout` is specified by either endpoint in its transport parameters, the connection is silently closed and its state is discarded when it remains idle for longer than the minimum of the `max_idle_timeout` value advertised by both endpoints.

Each endpoint advertises a `max_idle_timeout`, but the effective value at an endpoint is computed as the minimum of the two advertised values (or the sole advertised value, if only one endpoint advertises a non-zero value). By announcing a `max_idle_timeout`, an endpoint commits to initiating an immediate close if it abandons the connection prior to the effective value.

An endpoint restarts its idle timer when a packet from its peer is received and processed successfully. An endpoint also restarts its idle timer

when sending an ack-eliciting packet if no other ack-eliciting packets have been sent since last receiving and processing a packet. Restarting this timer when sending a packet ensures that connections are not closed after new activity is initiated.

To avoid excessively small idle timeout periods, endpoints **MUST** increase the idle timeout period to be at least three times the current Probe Timeout (PTO). This allows for multiple PTOs to expire, and therefore multiple probes to be sent and lost, prior to idle timeout.

## 2.1 Liveness Testing

An endpoint that sends packets close to the effective timeout risks having them be discarded at the peer, since the idle timeout period might have expired at the peer before these packets arrive.

An endpoint can send a PING or another ack-eliciting frame to test the connection for liveness if the peer could time out soon, such as within a PTO. This is especially useful if any available application data cannot be safely retried. Note that the application determines what data is safe to retry.

## 2.2 Deferring Idle Timeout

An endpoint might need to send ack-eliciting packets to avoid an idle timeout if it is expecting response data but does not have or is unable to send application data.

An implementation of QUIC might provide applications with an option to defer an idle timeout. This facility could be used when the application wishes to avoid losing state that has been associated with an open connection but does not expect to exchange application data for some time. With this option, an endpoint could send a PING frame periodically, which will cause the peer to restart its idle timeout period. Sending a packet containing a PING frame restarts the idle timeout for this endpoint also if this is the first ack-eliciting packet sent since receiving a packet. Sending a PING frame causes the peer to respond with an acknowledgment, which also restarts the idle timeout for the endpoint.

Application protocols that use QUIC **SHOULD** provide guidance on when deferring an idle timeout is appropriate. Unnecessary sending of PING frames could have a detrimental effect on performance.

A connection will time out if no packets are sent or received for a period longer than the time negotiated using the `max_idle_timeout` transport parameter. However, state in middleboxes might time out earlier than that. Though REQ-5 in [RFC4787](#) recommends a 2-minute timeout interval, experience shows that sending packets every 30 seconds is necessary to prevent the majority of middleboxes from losing state for UDP flows [[GATEWAY](#)].

TCP supports the keep-alive feature, and Linux provides a socket option `SO_KEEPALIVE` to enable this feature. HTTP also defines a header called `Keep-Alive`, which inherently relies on TCP keep-alive.

### 3 Stateless Reset

A stateless reset is provided as an option of last resort for an endpoint that does not have access to the state of a connection. A crash or outage might result in peers continuing to send data to an endpoint that is unable to properly continue the connection. An endpoint MAY send a Stateless Reset in response to receiving a packet that it cannot associate with an active connection.

A stateless reset is not appropriate for indicating errors in active connections. An endpoint that wishes to communicate a fatal connection error MUST use a `CONNECTION_CLOSE` frame if it is able.

To support this process, an endpoint issues a stateless reset token, which is a 16-byte value that is hard to guess. If the peer subsequently receives a Stateless Reset, which is a UDP datagram that ends in that stateless reset token, the peer will immediately end the connection.

A stateless reset token is specific to a connection ID. An endpoint issues a stateless reset token by including the value in the Stateless Reset Token field of a `NEW_CONNECTION_ID` frame. Servers can also issue a `stateless_reset_token` transport parameter during the handshake that applies to the connection ID that it selected during the handshake. These exchanges are protected by encryption, so only client and server know their value. Note that clients cannot use the `stateless_reset_token` transport parameter because their transport parameters do not have confidentiality protection.

Tokens are invalidated when their associated connection ID is retired via a `RETIRE_CONNECTION_ID` frame.

#### 3.1 Stateless Reset Packet

An endpoint that receives packets that it cannot process sends a packet in the following layout:

```
Stateless Reset {  
    Fixed Bits (2) = 1,  
    Unpredictable Bits (38..),  
    Stateless Reset Token (128),  
}
```

This design ensures that a Stateless Reset is – to the extent possible – indistinguishable from a regular packet with a short header.

A Stateless Reset uses an entire UDP datagram, starting with the first two bits of the packet header. The remainder of the first byte and an arbitrary number of bytes following it are set to values that SHOULD be indistinguishable from random. The last 16 bytes of the datagram contain a stateless reset token.

To entities other than its intended recipient, a Stateless Reset will appear to be a packet with a short header. For the Stateless Reset to appear as a

valid QUIC packet, the Unpredictable Bits field needs to include at least 38 bits of data (or 5 bytes, less the two fixed bits).

The resulting minimum size of 21 bytes does not guarantee that a Stateless Reset is difficult to distinguish from other packets if the recipient requires the use of a connection ID. To achieve that end, the endpoint SHOULD ensure that all packets it sends are at least 22 bytes longer than the minimum connection ID length that it requests the peer to include in its packets, adding PADDING frames as necessary. This ensures that any Stateless Reset sent by the peer is indistinguishable from a valid packet sent to the endpoint. An endpoint that sends a Stateless Reset in response to a packet that is 43 bytes or shorter SHOULD send a Stateless Reset that is one byte shorter than the packet it responds to.

These values assume that the stateless reset token is the same length as the minimum expansion of the packet protection AEAD. Additional unpredictable bytes are necessary if the endpoint could have negotiated a packet protection scheme with a larger minimum expansion.

An endpoint MUST NOT send a Stateless Reset that is three times or more larger than the packet it receives to avoid being used for amplification. Section §3.4 describes additional limits on Stateless Reset size.

Endpoints MUST discard packets that are too small to be valid QUIC packets. To give an example, with the set of AEAD functions defined in *mini RFC: Packet Protection*, short header packets that are smaller than 21 bytes are never valid.

Endpoints MUST send Stateless Resets formatted as a packet with a short header. However, endpoints MUST treat any packet ending in a valid stateless reset token as a Stateless Reset, as other QUIC versions might allow the use of a long header.

An endpoint MAY send a Stateless Reset in response to a packet with a long header. Sending a Stateless Reset is not effective prior to the stateless reset token being available to a peer. In this QUIC version, packets with a long header are only used during connection establishment. Because the stateless reset token is not available until connection establishment is complete or near completion, ignoring an unknown packet with a long header might be as effective as sending a Stateless Reset.

An endpoint cannot determine the Source Connection ID from a packet with a short header; therefore, it cannot set the Destination Connection ID in the Stateless Reset. The Destination Connection ID will therefore differ from the value used in previous packets. A random Destination Connection ID makes the connection ID appear to be the result of moving to a new connection ID that was provided using a NEW\_CONNECTION\_ID frame.

Using a randomized connection ID results in two problems:

- The packet might not reach the peer. If the Destination Connection ID is critical for routing toward the peer, then this packet could be incorrectly routed. This might also trigger another Stateless Reset in response; see §3.4. A Stateless Reset that is not correctly routed is an ineffective error detection and recovery mechanism. In this case,

Note that stateless reset packets are not encrypted. In fact, they are impossible to be encrypted as states are lost. But we must make them look like encrypted packets.

endpoints will need to rely on other methods – such as timers – to detect that the connection has failed.

- The randomly generated connection ID can be used by entities other than the peer to identify this as a potential Stateless Reset. An endpoint that occasionally uses different connection IDs might introduce some uncertainty about this.

### 3.2 Detecting a Stateless Reset

An endpoint detects a potential Stateless Reset using the trailing 16 bytes of the UDP datagram. An endpoint remembers all stateless reset tokens associated with the connection IDs and remote addresses for datagrams it has recently sent. This includes Stateless Reset Token field values from `NEW_CONNECTION_ID` frames and the server's transport parameters but excludes stateless reset tokens associated with connection IDs that are either unused or retired. The endpoint identifies a received datagram as a Stateless Reset by comparing the last 16 bytes of the datagram with all stateless reset tokens associated with the remote address on which the datagram was received.

This comparison can be performed for every inbound datagram. Endpoints *MAY* skip this check if any packet from a datagram is successfully processed. However, the comparison *MUST* be performed when the first packet in an incoming datagram either cannot be associated with a connection or cannot be decrypted.

An endpoint *MUST NOT* check for any stateless reset tokens associated with connection IDs it has not used or for connection IDs that have been retired.

If the last 16 bytes of the datagram are identical in value to a stateless reset token, the endpoint *MUST* enter the draining period and not send any further packets on this connection.

### 3.3 Calculating a Stateless Reset Token

The stateless reset token *MUST* be difficult to guess. In order to create a stateless reset token, an endpoint could randomly generate `[RANDOM]` a secret for every connection that it creates. However, this presents a coordination problem when there are multiple instances in a cluster or a storage problem for an endpoint that might lose state. Stateless reset specifically exists to handle the case where state is lost, so this approach is suboptimal.

A single static key can be used across all connections to the same endpoint by generating the proof using a pseudorandom function that takes a static key and the connection ID chosen by the endpoint as input. An endpoint could use HMAC [\[RFC2104\]](#) (for example, `HMAC(static_key, connection_id)`) or the HMAC-based Key Derivation Function (HKDF) [\[RFC5869\]](#) (for example, using the static key as input keying material, with the connection ID

A stateless reset packet will close a connection immediately, which makes it a perfect target of Denial-of-Service attack.

as salt). The output of this function is truncated to 16 bytes to produce the stateless reset token for that connection.

An endpoint that loses state can use the same method to generate a valid stateless reset token. The connection ID comes from the packet that the endpoint receives.

This design relies on the peer always sending a connection ID in its packets so that the endpoint can use the connection ID from a packet to reset the connection. An endpoint that uses this design **MUST** either use the same connection ID length for all connections or encode the length of the connection ID such that it can be recovered without state. In addition, it cannot provide a zero-length connection ID.

Revealing the stateless reset token allows any entity to terminate the connection, so a value can only be used once. This method for choosing the stateless reset token means that the combination of connection ID and static key **MUST NOT** be used for another connection. A denial-of-service attack is possible if the same connection ID is used by instances that share a static key or if an attacker can cause a packet to be routed to an instance that has no state but the same static key. A connection ID from a connection that is reset by revealing the stateless reset token **MUST NOT** be reused for new connections at nodes that share a static key.

The same stateless reset token **MUST NOT** be used for multiple connection IDs. Endpoints are not required to compare new values against all previous values, but a duplicate value **MAY** be treated as a connection error of type `PROTOCOL_VIOLATION`.

Note that Stateless Resets do not have any cryptographic protection.

### 3.4 Looping

The design of a Stateless Reset is such that without knowing the stateless reset token it is indistinguishable from a valid packet. For instance, if a server sends a Stateless Reset to another server, it might receive another Stateless Reset in response, which could lead to an infinite exchange.

An endpoint **MUST** ensure that every Stateless Reset that it sends is smaller than the packet that triggered it, unless it maintains state sufficient to prevent looping. In the event of a loop, this results in packets eventually being too small to trigger a response.

An endpoint can remember the number of Stateless Resets that it has sent and stop generating new Stateless Resets once a limit is reached. Using separate limits for different remote addresses will ensure that Stateless Resets can be used to close connections when other peers or connections have exhausted limits.

A Stateless Reset that is smaller than 41 bytes might be identifiable as a Stateless Reset by an observer, depending upon the length of the peer's connection IDs. Conversely, not sending a Stateless Reset in response to a small packet might result in Stateless Resets not being useful in detecting cases of broken connections where only very small packets are sent; such

Stateless reset is useful only when the other endpoint could recognize that it receives a stateless reset. Loop is possible but rare. Note that stateless reset reacts to unknown packets. The endpoint sending these packets is very likely to retain states. If not why does it even send packets?

failures might only be detected by other means, such as timers.

## **4 Packets and Frames**

### **4.1 Packets**

There is no new type of packets introduced in this RFC.

### **4.2 Frames**

There is no new type of frames introduced in this RFC.

## **5 Application Protocol**

There is no new application protocol defined in this mini RFC.